The journey of a Snapchat message - Traffic Capture

Snapchat Hacking & Masking

Zoe J.D.M. Spyrou
Student Number: 452280
Fontys HBO-ICT
Semester 4 OL

Table of Contents

Edit Log

Date	Task
3/11/2025	Finished first version
4/11/2025	Added cover page, TOC, Methodology + Uploaded to Canvas + Uploaded relevant logs

> **Introduction**
>
> The police uses Snapchat to talk to suspected individuals, through fake/controlled accounts portraying to be underage children, using AI generated images. The goal of our project is to find a way to mask the activity of the police and extract the IP address of the suspects, so that they can then use it to find out the location of the suspect.
>
> Masking is defined as:
>
> - Make it appear as if messages sent on Snapchat are being sent on a mobile device, not a browser, as Snapchat notifies users from where messages are being sent through
> - Images that are being sent need to appear as images that were taken through Snapchat, and not as images directly uploaded through a camera roll, as those appear different in Snapchat
> - Making an account look relatively active (snapscore not 0, aka not 0 messages sent in total, as it appears that the account has not been used yet)
> - Not notifying the suspect of screenshots taken of the conversation, needed for collection of evidence
>
> All research and testing is done on accounts that are either owned by us (the team that is working on the project) or the stakeholders (police Rotterdam, Fontys coaches). No non-consenting parties are used in testing/experimenting. These efforts are not to be used in any scope outside the project, as that goes against the policy of Snapchat. All evidence found is to be kept between the Stakeholders and us.

Context

In this document I analyze the "journey" a message sent on Snapchat goes through before reaching the receivers' device, for both messages sent on the mobile version of the app.

The theory is that the app and web version use different API endpoints, and thus masking Snpachat's web version, requires routing data through the mobile version.

Objective

Learn how to capture the information mentioned in the context and capture it. Analyze the results and see if they are usable in the context of the project.

Scope

Controlled testing on personally owned devices or consenting test accounts.

Research Methodology

Based on the DOT-Framework, the following methodologies have been utilized:

Method	What/How/Why
Available Product Analysis	Researched several tools that were available to analyze the HTTPS traffic and made a table to see which one better suited my needs for this project.
Multi-Criteria Decision Making	Same idea as available product analysis, leaning more on the comparison between the tools researched.
Literature Study	Reviewed a lot of online documentation and read about ways to use tools included in my testing, as well as feats of other people who tried to do something similar, including Snapchat and other social media applications.
Expert Interview	Set up a meeting for my entire group with Tom Meulensteen to discuss the current state of the project. Mentioned the methods I had tried to capture traffic and the expert could not find any alternatives either.
Stakeholder Analysis	Made sure that the solution working on includes the stakeholder's wishes, aka, the solution has to be working on a computer.

How to gather the information

The initial thought here was to use Wireshark to capture packets that are being sent and received from both the devices (Android Phone and Laptop). This however poses a level of difficulty, as Wireshark captures all network traffic on an interface, meaning also a lot of data that has nothing to do with the research I am doing, which will make any data captured harder to understand. Tools like tcpdump can be used with it, as well as making Wiresharks' setup very targeted, but it makes more sense to use a tool that is specifically designed to do what I need, which is capture HTTP/HTTPS traffic and discover API calls.

Moreover, seeing as Snapchat is a big corporation, is highly unlikely that they send unencrypted data. It will most likely be encrypted. A proxy server to route traffic through is a much better solution, to capture and decrypt data. Essentially, performing a Man-in-the-Middle attack, on my own device, to see the full HTTP/HTTPS requests and responses, including headers, API endpoints etc.

As we will be testing the flow of a message, it might also yield interesting results to log what happens during log in on a device. This will yield more data for me to analyze and theorize about masking methods.

Below is a list of tools that work a little more specific rather than Wireshark's catch-all.

1. Charles Proxy

"Charles is an HTTP proxy / HTTP monitor / Reverse Proxy that enables a developer to view all of the HTTP and SSL / HTTPS traffic between their machine and the Internet. This includes requests, responses and the HTTP headers (which contain the cookies and caching information)." (Cactuslab, n.d.)

- Has an interface
- SSL Proxying, view SSL requests and responses in plain text
- Edit requests to test different inputs
- Breakpoints to intercept and edit requests or responses

2. mitmproxy

"mitmproxy is your swiss-army knife for debugging, testing, privacy measurements, and penetration testing. It can be used to intercept, inspect, modify and replay web traffic such as HTTP/1, HTTP/2, HTTP/3, WebSockets, or any other SSL/TLS-protected protocols. You can prettify and decode a variety of message types ranging from HTML to Protobuf, intercept specific messages on-the-fly, modify them before they reach their destination, and replay them to a client or server later on" (*Mitmproxy - an Interactive HTTPS Proxy*, n.d.)

- Has a web interface, as well as command line
- Pyton API, can be used to modify messages, redirect traffic, or other custom commands
- Community contributed mitmproxy addons

3. Burp Suite

"Notable capabilities in this suite include features to proxy web-crawls (Burp Proxy), log HTTP requests/responses (Burp Logger and HTTP History), capture/intercept in-motion HTTP requests (Burp Intercept), and aggregate reports which indicate weaknesses (Burp Scanner). This software uses a built-in database containing known-unsafe syntax patterns and keywords to search within captured HTTP requests/responses." (Wikipedia contributors, 2025)

- Target URLs' site maps can be captured either through automatic or manual web-crawling
- Repeats captured HTTP requests, allowing custom changes to request variables
- Automates text decoding
- Analyzes an application-generated token variable across repeated HTTP requests to determine pseudorandomness predictability strength
- Java scripts to create custom HTTP request/response

Analysis en recommendation for this project

These tools all have a learning curve to them, and to effectively manage time, a choice is needed to be made between which tool will be used. A table here will analyze the strengths and weaknesses based on the project's needs to make an informed choice.

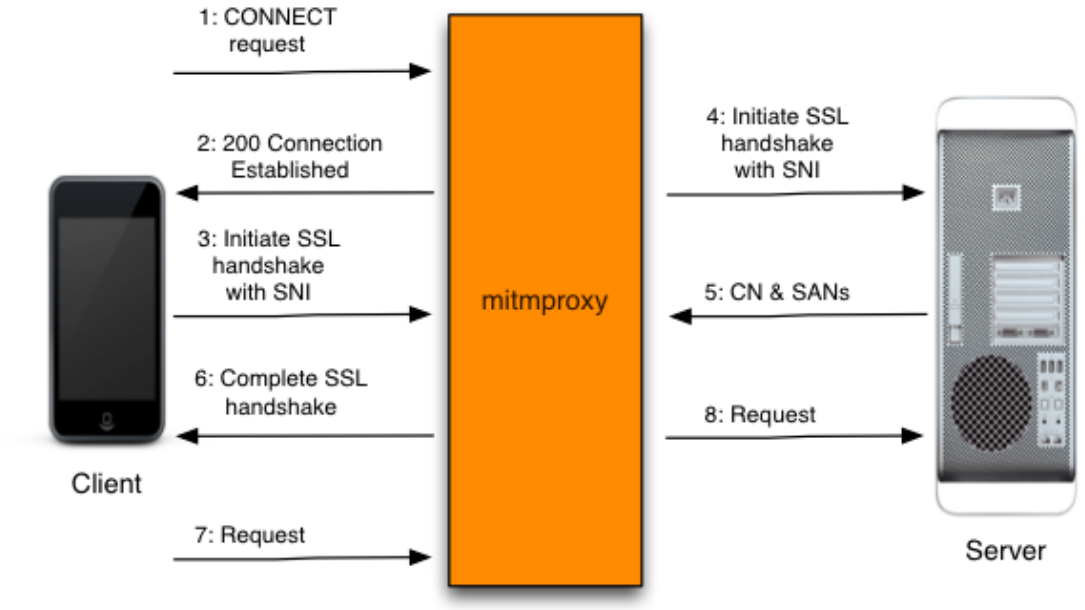
	Wireshark	Charles Proxy	mitmproxy	Burp Suite
Free	++	- ($50)	++	+ (community edition)
Easy Learning Curve	- (complex)	++	+ (CLI focused, has web GUI)	+
Mobile device setup	+ (Requires network config)	++	++	++
Filtering	+	++	++	++
Editing requests/responses	-	++	++	++
Can be integrated in script	+	-	++ (Python library)	+

Based on this table, "mitmproxy" stands out as the optimal tool for this project. Its combination of being fully free and open-source and its native scriptability via its Python API makes it uniquely suited for the project's goals. While its command-line interface presents a slightly steeper initial learning curve than Charles Proxy, its integrated web interface (mitmweb) provides an accessible way to perform the initial traffic analysis and discovery.

The ability to eventually write custom Python scripts is the decisive advantage. This feature is the key to developing a functional "masking" system that can automatically modify outbound requests. Such a script could, in theory, alter the web client's traffic in real-time to spoof the mobile client's headers (like `User-Agent` and `X-Snapchat-Client-Properties`) and reformat image uploads to match the mobile app's API calls. This directly addresses the core objective of making police web activity on Snapchat indistinguishable from that of a genuine mobile user.

Using mitmproxy

A quick visual of of mitmproxy works:



mitmproxy proxied HTTPS flow.png

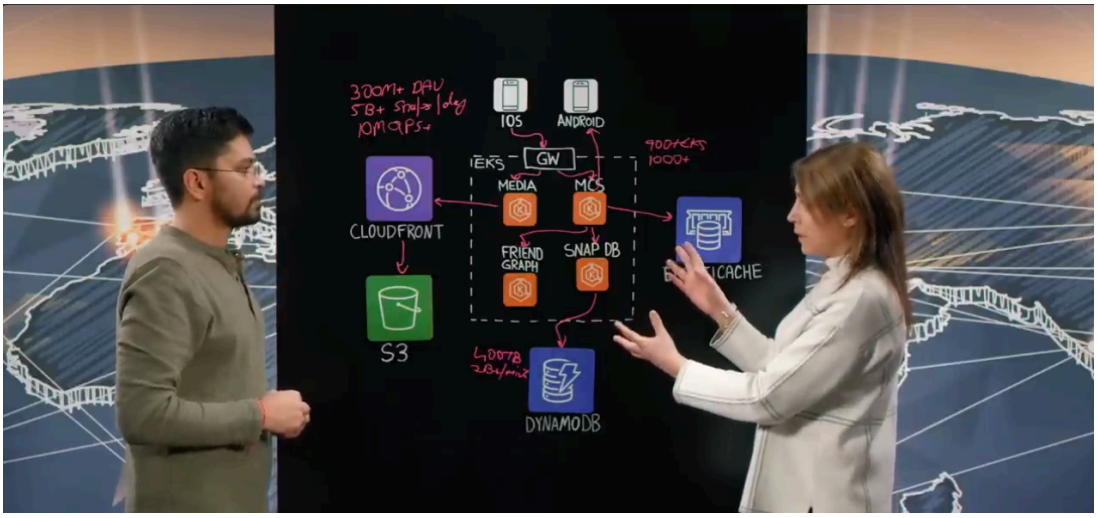
Quoting directly from the documentation:

```
1. The client makes a connection to mitmproxy, and issues an
HTTP CONNECT request.
2. Mitmproxy responds with a `200 Connection Established`,
as if it has set up the CONNECT pipe.
3. The client believes it’s talking to the remote server,
and initiates the TLS connection. It uses SNI to indicate
the hostname it is connecting to.
4. Mitmproxy connects to the server, and establishes a TLS
connection using the SNI hostname indicated by the client.
5. The server responds with the matching certificate, which
contains the CN and SAN values needed to generate the
interception certificate.
6. Mitmproxy generates the interception cert, and continues
the client TLS handshake paused in step 3.
7. The client sends the request over the established TLS
connection.
8. Mitmproxy passes the request on to the server over the
TLS connection initiated in step 4.
```

The goal is to set up proxy on my laptop that routes all traffic from my phone through it, so that I can see the content of the requests and responses from Snapchat, before those are actually sent out to the server/come in to the phone.

Information found

Through searching about AWS and Snapchat, I had found quite an informative video, which featured the journey of a Snap (message, image, etc), from about 3 years ago, when the web version did not exist yet. In the screenshot a diagram is depicted of the journey of a Snap:



Amazon Web Services, 2022.png

I can only assume that a lot has changed from that time, now that a web version also exists, with other checks to detect the device used. They did not share any information about API's used of course.

First attempt

Environment: Firefox browser (version 134.0 (64-bit)) on Linux Mint, running mitmproxy + Physical Android phone (model: One Plus 8 Pro), not rooted, OS: Oxygen OS 13

I had set up `mitmproxy` on my laptop, added the certificate to my browser, and was able to see all traffic going in and out of my device. In the settings tab of `mitmproxy` there was an option to set a Wireguard Server up to proxy data from another device on the same network. I had tried this, and while I was able to capture some traffic from my phone, it was not a whole lot of success. I had to manually accept risks of "expired certificates" to visit a website but this is of course not an option with apps.

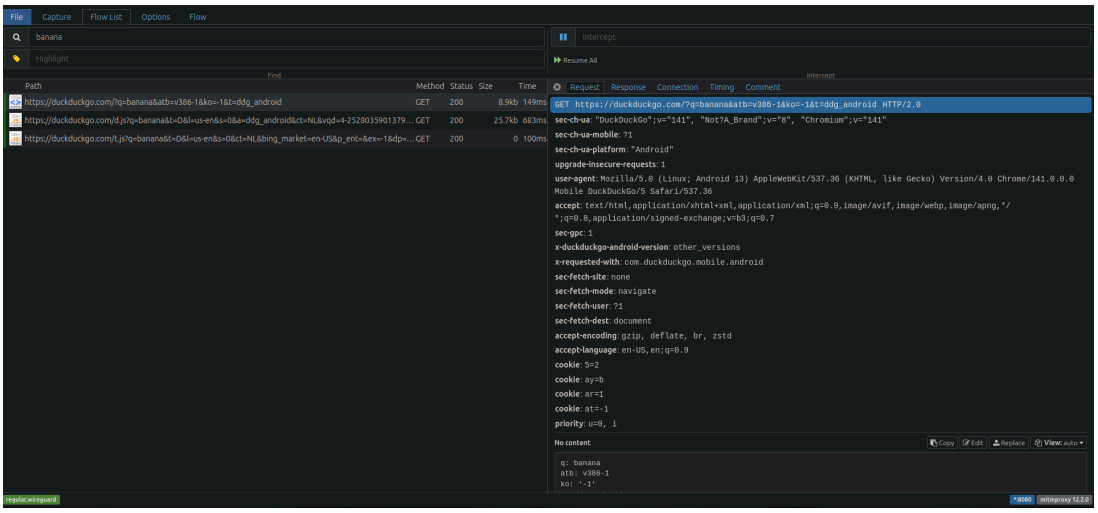


image-67.png

After this attempt, I tried to manually set up the proxy server on my phone. To do this, in my WiFi settings, I had to edit the current connection as set the proxy server to the IP of my laptop and the port to any random port that I would use, in my case port 8080.

Moreover, from logging I could also see that the issue remained the certificate. In order to fix this issue, I had to move on to testing on a rooted device, where I could import certificates that are system trusted, rather than user trusted.

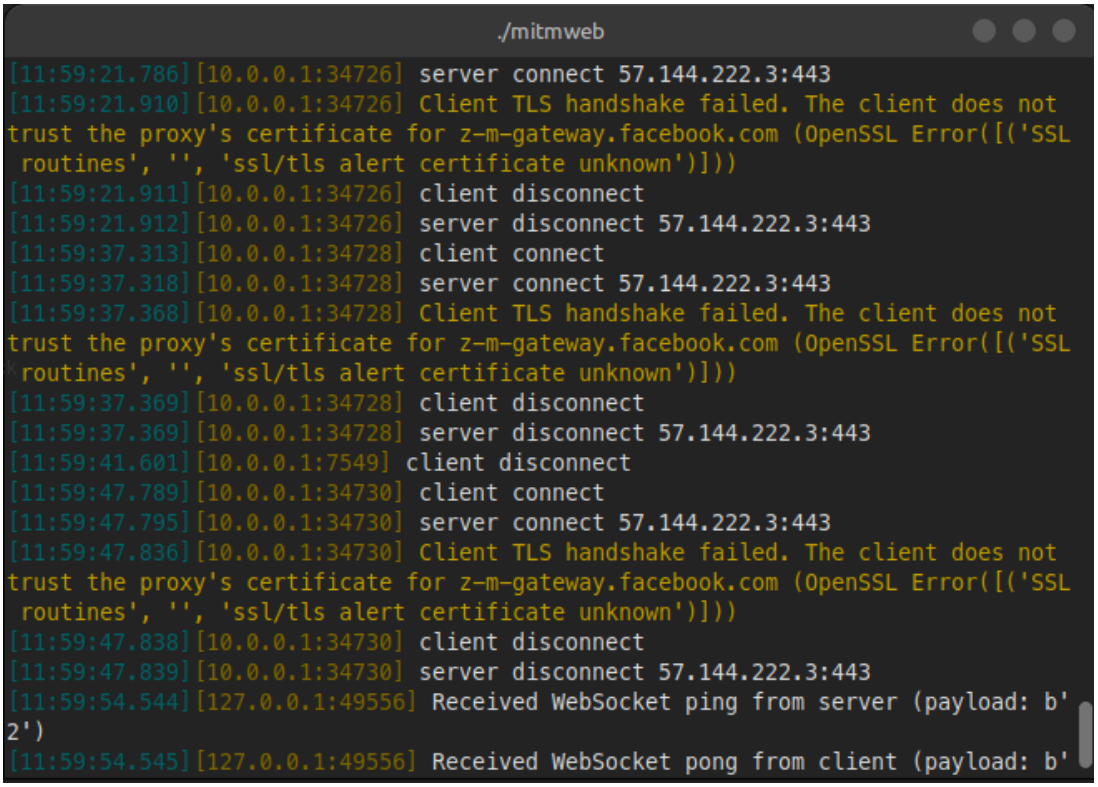


image-68.png

If the certificate issue were to be fixed, I could capture Snapchat's traffic. At the moment I can see for example that the Snapchat app was launched, but I cannot do anything further with this data, nor it is much help.

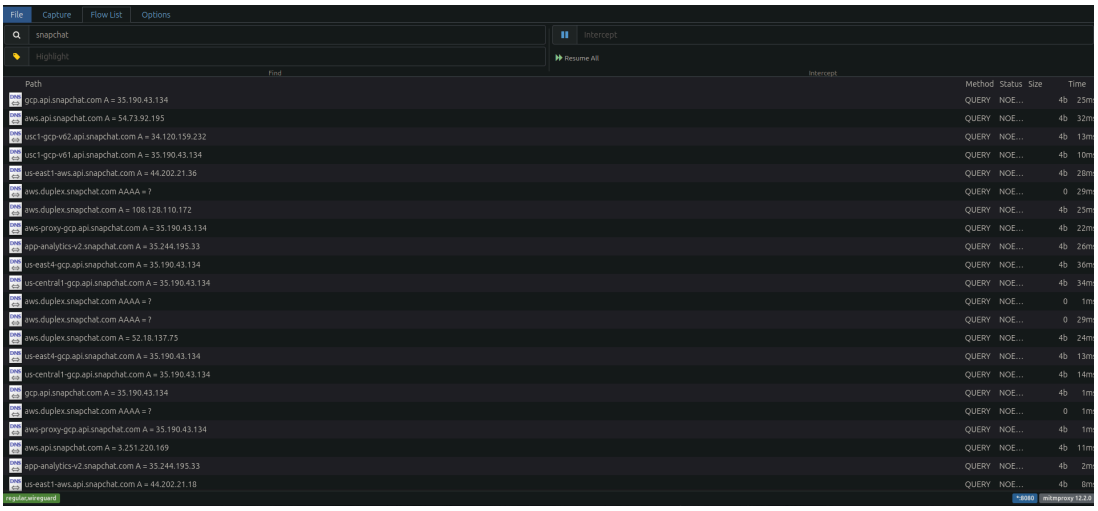


image-69.png

For a quick comprison, this is what it looks like when I access the web version of Snapchat (only the last entry, that is currently open in the Requests tab, notice the status 101):

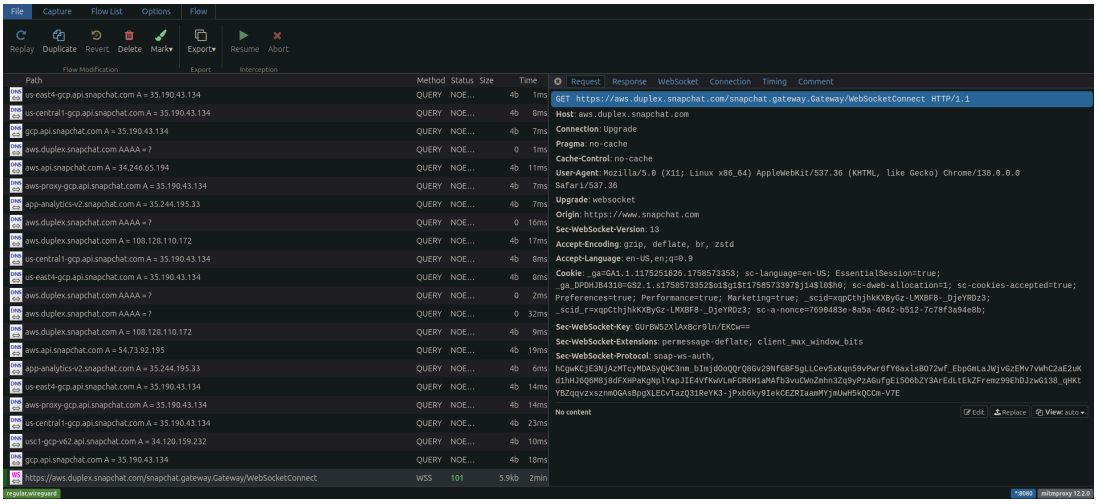


image-70.png

Second Attempt

Environment: Firefox browser (version 134.0 (64-bit)) on Linux Mint, running mitmproxy + Physical Android phone (model: One Plus 8 Pro), rooted with Magisk, OS: Oxygen OS 13

This time I followed mostly the same steps as above by setting up the proxy on the phone, but prior to doing that, I had to take a look at masking the fact that the device was rooted, as that was an issue last time with Snapchat. I removed all excess Magisk modules I had from my last time working with the rooted phone, and followed a tutorial I had found on YouTube (TechyNoob, 2025), that showed the modules required for passing the Play Integrity checks on an Android device (used to be called SafetyNet, hence in the sources below it will also be mentioned by that name).

Those modules were:

- Integrity Box
- Play Integrity Fork
- Tricky Store

After installing and rebooting the device, I used an app called "Play Integrity API Checker". In short, the Play Integrity API was made by Google and checks if user activity on a device is coming from a genuine app downloaded through the play store, running on a genuine and certified Android device. It detects if an environment is simulated, rooted, or otherwise tampered with. This is why for example you are not able to do any banking activities on a rooted Android device, as the app it's self performs the check, the check is gathered by an API, and the backend of the app responds with an error, to protect the user from possible fraudulent activity.

After verifying the modules worked, I installed Snapchat on the device from the official Play Store (this time without issues), and I was able to log in and use the app as I normally would and send and receive messages.

After testing the above, I activated the proxy again to see if I could get any more information this time from the requests and calls. At first I checked with just using my phone's browser, to see if I would yet again get certificate issues, but this time there was no problem, as the certificate was now system trusted.

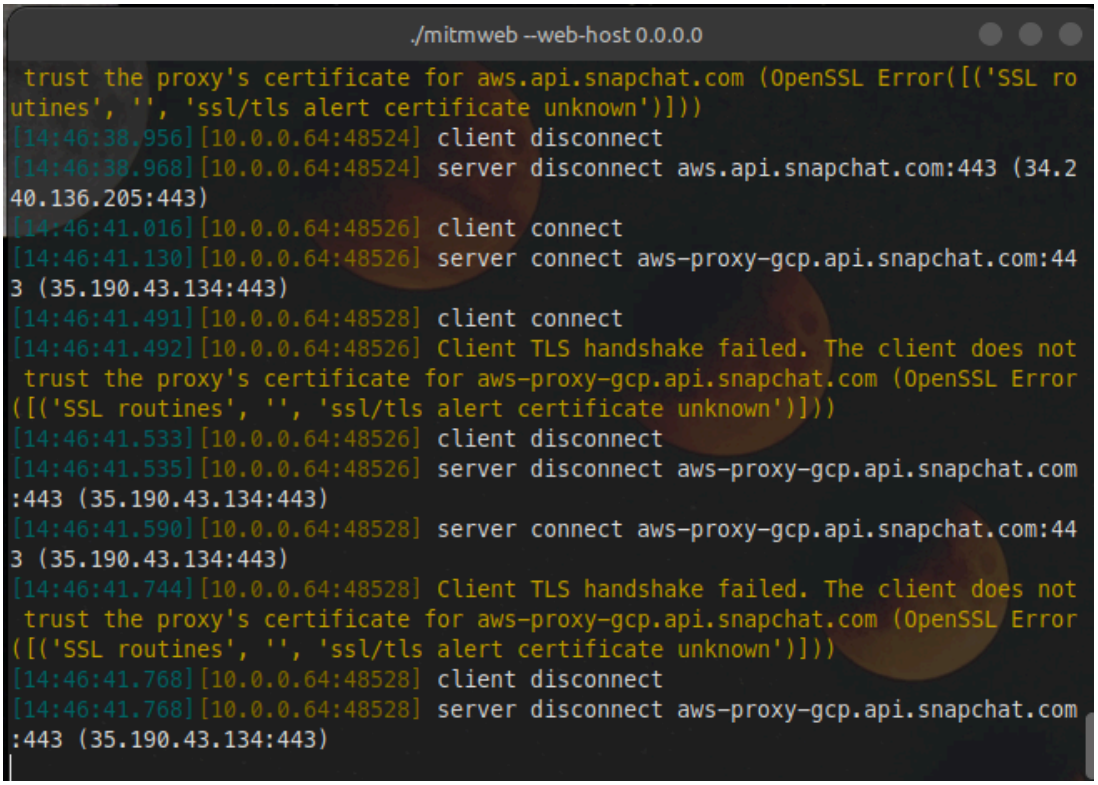


image-72.png

Sending messages or receiving them on Snpachat was still not possible, despite the certificate being system trusted. My certificate was still not trusted by Snapchat themselves, who use SSL pinning to ensure only verified certificates are accepted.

Third attempt

Environment: Firefox browser (version 134.0 (64-bit)) on Linux Mint, running mitmproxy, Frida + Physical Android phone (model: One Plus 8 Pro), rooted with Magisk, hosting Frida-server, OS: Oxygen OS 13

It was now time to learn more about SSL pinning and unpinning. The rooted Android device is once again used for this scenario.

By looking through Reddit, I discovered Frida (*Frida • a World-class Dynamic Instrumentation Toolkit*, n.d.). Quoting the documentation of Frida directly:

```
It’s Greasemonkey for native apps, or, put in more technical terms, it’s a dynamic code instrumentation toolkit. It lets you inject snippets of JavaScript or your own library into native apps on Windows, macOS, GNU/Linux, iOS, watchOS, tvOS, Android, FreeBSD, and QNX. Frida also provides you with some simple tools built on top of the Frida API.
```

With Frida thus, I could insert my own little code snippets in an app as it launched on my phone. To use it, I had download their toolkit on my laptop and set up a Frida-server on my phone, using `adb`. To use it then, a script was downloaded from their website, that claimed to do SSL unpinning.

This should launch the app specified with the correct script attached to it at startup:

```
frida -U -f com.snapchat.android -l snapchat_unpin.js --no-pause
```

This unfortunately did not work. I renamed the Frida host on the Android device to something more obscure, as there are apps that are actively protecting themselves against Frida, to no success.

I wanted to verify that I was following the correct steps and that this was not just a "me-issue". I downloaded HTTP Toolkit after reccomnedation from Reddit, which automates the process of setting up the server, the client, and

the script imports. All I had to do was install their certificate. This did not work either, so using Frida at this point was not an option anymore.

Multiple different scripts have been tested, but none had any succcess.

Fourth Attempt

Environment: Firefox browser (version 134.0 (64-bit)) on Linux Mint, running mitmproxy + Physical Android phone (model: One Plus 8 Pro), rooted with Magisk, OS: Oxygen OS 13

Another attempt had been made to track Snapchat's HTTPS traffic on the the Android phone. This time the process was similar again to the second attempt (using just mitmproxy). For this attempt, a Magisk module has been used additionally, named `Zygisk SSL Unpinning` M0szy (n.d.).

The module can be configured to attach to specific apps on a phone, by adding a new object to the config file, like this:

```
{
    "targets": [
        {
            "pkg_name" : "com.target.package",
            "enable": true,
            "start_safe": true,
            "start_delay": 1000
        }
    ]
}
```

After installing and rebooting the phone, I checked the traffic again that was visible in the mitm web interface. Thid time instead of no response at all from Snapchat, I got something. Logs will be uploaded alongside document. I asked AI to sumarize and explain the 2 most prevalent logs for me, as I did not fully understand what I was reading:

*"These captures show **mobile Snapchat traffic** using **gRPC over HTTP/2** (`content-type: application/grpc`, `ALPN: h2`) to `aws.api.snapchat.com`, with clear RPCs like `PushNotificationDataRegistryService/RegisterDevice` (containing an FCM-style token) and `sparta/.../DeltaSync`. Requests include a long `x-snap-access-token` header and cronet-based user-agents (`grpc-c++`, `Snapchat/<version>`); payloads are binary Protobufs (hence the*

`!binary` / `!sint` output) and TLS is modern (TLS 1.3, DigiCert cert). Timestamps in the responses let you correlate actions (send/open/receive) and the presence of push-token and sync RPCs strongly indicates mobile-specific endpoints and flows that differ from the web client (which shows websocket/HTTP patterns).

In your testing you hit expected protections: app pinning and integrity checks prevented casual interception until you used more intrusive methods on a test device; after overcoming pinning you were able to see the gRPC traffic (RegisterDevice/DeltaSync)"

`gPRC` stands for Remote Procedure Call and is a framework that can be run in any environment. It connects services across mobile applications and browsers to backend services. (*GRPC*, n.d.)

Conclusion and Next Steps

The testing shows that, while it is possible to observe portions of Snapchat's HTTPS traffic on a rooted Android device, there is no simple or reliable way to reproduce those results from a web browser. Snapchat's certificate pinning, gRPC-based mobile communication, and integrity checks make web-based traffic manipulation impractical and brittle; the logs obtained do not provide enough information to form a credible exploit hypothesis without a high risk of app malfunction or detection.

Given that Snapchat can now run on a suitably prepared test device, the project will shift focus to camera masking. The working assumption is that spoofing camera hardware properties may reduce the need to mask the device type itself. The solution should still work when interacting with a computer, as tools such as `adb` and `scrcpy` can be used to manage and mirror the device reliably, provided the connection remains stable. The immediate goal is to determine whether a camera-masking approach can survive the phone's integrity checks and Snapchat's own verification mechanisms.

Sources

Cactuslab. (n.d.). *Charles Web Debugging Proxy • HTTP monitor / HTTP proxy / HTTPS & SSL proxy / Reverse proxy*. Copyright (C) 2025 XK72 Ltd. https://www.charlesproxy.com/

mitmproxy - an interactive HTTPS proxy. (n.d.). https://www.mitmproxy.org/

Wikipedia contributors. (2025, June 29). *Burp Suite*. Wikipedia. https://en.wikipedia.org/wiki/Burp_Suite
How mitmproxy works. (n.d.). https://docs.mitmproxy.org/stable/concepts/how-mitmproxy-works/

Amazon Web Services. (2022, July 26). *SNAP: Journey of a snap on Snapchat using AWS* [Video]. YouTube. https://www.youtube.com/watch?v=Cgv0kfp_6xQ

Intercepting Android HTTP. (n.d.). https://httptoolkit.com/docs/guides/android/

TechyNoob. (2025, September 15). *Fix SafetyNet/Play Integrity on ANY rooted Android (2025)*
[Video]. YouTube. https://www.youtube.com/watch?v=9hoLkM0wV_Y

Frida • A world-class dynamic instrumentation toolkit. (n.d.). Frida • a World-class Dynamic Instrumentation Toolkit. https://frida.re/

M0szy. (n.d.). *GitHub - m0szy/Zygisk-SSL-Unpinning: ssl unpinning with zygisk*. GitHub. https://github.com/m0szy/Zygisk-SSL-Unpinning

GRPC. (n.d.). gRPC. https://grpc.io/